LAPORAN PRAKTIKUM HDFS



Dosen Pengampu:

Ersha Aisyah Elfaiz, S.Si., M.T.I.

Disusun oleh:

Muhammad Anwar Ismail (24051204161)

Program Studi Teknik Informatika

Fakultas Teknik

Universitas Negeri Surabaya

2026

DAFTAR ISI

DAFTAR ISI ..... **i**
DAFTAR GAMBAR **1**
BAB I PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Tujuan Praktikum 3
BAB II DASAR TEORI **5**
2.1 Paradigma Sistem Terdistribusi 5
2.2 Prinsip Dasar Sistem Terdistribusi 5
2.3 Konsep Apache Hadoop & Arsitektur HDFS 6
2.4 NameNode, DataNode, Replication Factor & Alur Penulisan Data 6
BAB III PEMBAHASAN **7**
3.1 Langkah-Langkah Praktikum dan Command 7
3.2 Hasil Eksekusi dan Screenshot 11
3.3 Jawaban Pertanyaan Analisis 27
BAB IV KESIMPULAN **30**
4.1 Kesimpulan 30
LAMPIRAN A COMMAND UTAMA **31**
LAMPIRAN B PORT YANG DIGUNAKAN **33**

DAFTAR GAMBAR

Gambar 3.2.1 - Menunjukkan struktur folder dan file yang berhasil dibuat.png 11
Gambar 3.2.2 - Validasi Docker Compose_1.png 12
Gambar 3.2.3 - Validasi Docker Compose_2.png 12
Gambar 3.2.4 - Proses Download Image_1.png 13
Gambar 3.2.5 - Proses Download Image_2.png 13
Gambar 3.2.6 - Menjalankan Cluster dan Verifikasi.png 14
Gambar 3.2.7 - Informasi Docker Desktop Container.png 15
Gambar 3.2.8 - Menampilkan Informasi Volume pada Docker.png 16
Gambar 3.2.9 - NameNode Web UI_1.png 17
Gambar 3.2.10 - NameNode Web UI_2.png 17
Gambar 3.2.11 - DataNode Web UI.png 18
Gambar 3.2.12 - Melihat isi root HDFS.png 19
Gambar 3.2.13 - Membuat folder Praktikum, data, hasil dan verifikasi output direktori.png 19
Gambar 3.2.14 - Salin file data.txt dari Windows ke dalam container NameNode.png 20
Gambar 3.2.15 - Pindahkan (upload) file tersebut dari container ke dalam HDFS.png 21
Gambar 3.2.16 - Tampilkan daftar file di direktori HDFS untuk memastikan file sudah ter-upload.png 21
Gambar 3.2.17 - Tampilkan isi file data.txt yang ada di HDFS.png 22
Gambar 3.2.18 - Buat salinan file (copy).png 23
Gambar 3.2.19 - Ubah nama (rename atau move) file salinan tersebut menjadi data-backup.txt.png 23
Gambar 3.2.20 - Hapus (delete) file salinan yang baru saja di-rename tersebut.png 24
Gambar 3.2.21 - Mengecek status DataNode dan kapasitas HDFS.png 25

Gambar 3.2.22 - Verifikasi akhir (Menampilkan seluruh isi folder mahasiswa).png 26
Gambar 3.2.23 - Mematikan Docker Container.png 26

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan volume data yang terus meningkat menuntut adanya lingkungan pengolahan data yang mampu bekerja secara terdistribusi, salah satunya melalui Apache Hadoop dengan komponen utamanya, *Hadoop Distributed File System* (HDFS). Pada praktikum ini, lingkungan Hadoop dibangun di atas sistem operasi Windows dengan memanfaatkan WSL 2 (*Windows Subsystem for Linux*) sebagai jembatan menuju lingkungan Linux, kemudian Docker Desktop digunakan untuk menjalankan Hadoop dalam bentuk *container* tanpa perlu instalasi manual yang rumit. Alur lingkungan tersebut dapat digambarkan sebagai berikut: Windows menjalankan WSL 2, di dalam WSL 2 berjalan Docker Desktop, dan di dalam Docker Desktop tersusun klaster Hadoop yang terdiri atas NameNode sebagai pengelola *metadata* dan DataNode sebagai penyimpan blok data.

Klaster pada praktikum ini sengaja dikonfigurasi hanya dengan satu DataNode agar ringan dan dapat dijalankan pada laptop mahasiswa dengan spesifikasi standar. Konfigurasi *single-node* ini ditujukan murni untuk kebutuhan pembelajaran dan pemahaman konsep dasar HDFS, bukan untuk merepresentasikan klaster produksi berskala besar yang umumnya memakai puluhan hingga ratusan DataNode sekaligus.

1.2 Tujuan Praktikum

Praktikum ini dilaksanakan dengan tujuan agar mahasiswa mampu:

1. Memahami konsep dasar dari arsitektur Apache Hadoop dan *Hadoop Distributed File System* (HDFS).
2. Membedakan fungsi serta peran krusial antara NameNode dan DataNode dalam klaster.
3. Melakukan instalasi dan konfigurasi lingkungan virtualisasi menggunakan WSL 2 dan Docker Desktop.
4. Menerapkan *deployment* klister Hadoop *single-node* berbasis *container* melalui Docker Compose.

5. Memantau kesehatan sistem terdistribusi melalui akses NameNode dan DataNode Web UI.
6. Mengeksekusi operasi dasar manajemen *file* pada ekosistem HDFS (mencakup pembuatan direktori, pengunggahan, pembacaan, penyalinan, pemindahan, hingga penghapusan *file*).
7. Membaca dan menganalisis laporan kondisi DataNode beserta utilitas kapasitas HDFS.

BAB II
DASAR TEORI

2.1 Paradigma Sistem Terdistribusi

. Sistem terdistribusi lahir dari kebutuhan menangani data berskala masif yang sudah tidak lagi realistis ditampung oleh satu mesin, sehingga pendekatan *scaling up* (menambah RAM, CPU, atau *disk* pada satu mesin tunggal) mulai ditinggalkan karena terbentur batas fisik perangkat keras, biaya yang melonjak eksponensial, dan risiko *Single Point of Failure*. Sebagai gantinya, diterapkan pendekatan *scaling out*, yaitu menghubungkan banyak mesin standar (*commodity hardware*) ke dalam satu *cluster* melalui jaringan, sehingga skalabilitas menjadi jauh lebih luas dan sistem lebih tangguh terhadap kegagalan. Berbeda dengan komputasi paralel konvensional yang mengandalkan *shared memory* dalam satu mesin dan rentan hancur total apabila *motherboard*-nya mati, komputasi terdistribusi menganut *distributed memory* dengan komunikasi antar-mesin melalui jaringan, sehingga bersifat *fault tolerant* karena kegagalan pada sebagian *node* dapat langsung diambil alih oleh *node* lain yang masih aktif.

2.2 Prinsip Dasar Sistem Terdistribusi

Keandalan sistem terdistribusi ditopang oleh tiga prinsip utama. Pertama, partisi data (*sharding*), yaitu memecah *dataset* raksasa menjadi blok-blok berukuran tetap (umumnya 128 MB) agar beban penyimpanan tersebar merata dan dapat dibaca secara paralel oleh banyak *node* sekaligus. Kedua, replikasi data, yaitu menggandakan setiap blok ke beberapa *node* dan rak fisik berbeda (standar industri tiga salinan) untuk mencegah kehilangan data, dengan prinsip lokalitas data yang mengutamakan memindahkan komputasi ke lokasi data alih-alih memindahkan data itu sendiri. Ketiga, toleransi kesalahan (*fault tolerance*), yang mencakup deteksi kegagalan melalui sinyal *heartbeat* serta pemulihan kegagalan melalui *checkpointing* dan *speculative execution*. Ketiga prinsip ini pada akhirnya berhadapan dengan Teorema CAP, yang menyatakan sebuah sistem hanya dapat mempertahankan dua dari tiga properti (*Consistency*, *Availability*, *Partition Tolerance*) saat terjadi gangguan jaringan, sehingga sistem Big Data umumnya

memilih filosofi BASE (*Basically Available*, *Soft State*, *Eventually Consistent*) dibanding filosofi ACID pada basis data relasional tradisional.

2.3 Konsep Apache Hadoop & Arsitektur HDFS

Apache Hadoop merupakan *framework open-source* untuk menyimpan dan memproses data berskala masif secara terdistribusi, sebagai penerapan langsung dari prinsip-prinsip sistem terdistribusi pada subbab sebelumnya. Ekosistemnya terdiri atas HDFS untuk penyimpanan, YARN untuk penjadwalan sumber daya, dan MapReduce untuk pemrosesan data. HDFS menganut topologi *Master-Worker* sebagai standar de facto analitik Big Data, di mana satu *node master* mengelola *metadata* tanpa menyentuh isi *file* asli, sementara sekumpulan *node* pekerja menyimpan blok data secara fisik dan mengeksekusi komputasi. Sejalan dengan Teorema CAP, HDFS lebih mengutamakan ketersediaan dan toleransi partisi dengan konsistensi bersifat *eventually consistent*, menjadikannya sesuai untuk analitik data masif yang menuntut fleksibilitas tinggi.

2.4 NameNode, DataNode, Replication Factor & Alur Penulisan Data

Arsitektur HDFS menganut sistem *Master-Slave*: NameNode berperan sebagai *Master* yang mengelola *metadata* dan struktur direktori, sedangkan DataNode berperan sebagai *Slave* yang menyimpan blok data secara fisik dan mengirim sinyal *heartbeat* setiap tiga detik. *Replication Factor* mengatur jumlah salinan cadangan tiap blok data, dengan standar tiga salinan pada lingkungan produksi, namun ditetapkan satu pada praktikum ini karena hanya tersedia satu DataNode aktif. Alur penulisan data berlangsung singkat: *client* melapor ke NameNode, menerima daftar DataNode tujuan, memecah *file* menjadi blok, mengirim blok pertama ke DataNode awal, lalu replikasi berlanjut otomatis secara berantai antar-DataNode hingga jumlah salinan sesuai *Replication Factor* tercapai alur inilah yang terjadi setiap kali perintah `hdfs dfs -put` dijalankan pada praktikum di BAB III.

BAB III
PEMBAHASAN

3.1 Langkah-Langkah Praktikum dan Command

Dalam pelaksanaan praktikum ini, terdapat beberapa tahapan instruksi yang dieksekusi melalui terminal PowerShell:

1. Pembuatan Direktori dan Konfigurasi Dasar

   Tahapan dimulai dengan pembuatan direktori kerja bigdata-hadoop di penyimpanan lokal. Di dalamnya, disusun file docker-compose.yml untuk orkestrasi *container*, serta file konfigurasi HDFS (core-site.xml dan hdfs-site.xml) yang ditempatkan pada folder conf.

   Berikut isi dari ketiga file:

   - core-site.xml

```
<?xml version="1.0"?>
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://namenode:8020</value>
</property>
</configuration>
```

   - hdfs-site.xml

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///tmp/hadoop-data/namenode</value>
  </property>
  <property>
```

```
    <name>dfs.datanode.data.dir</name>
    <value>file:///tmp/hadoop-data/datanode</value>
  </property>
  <property>
    <name>dfs.namenode.http-address</name>
    <value>0.0.0.0:9870</value>
  </property>
  <property>
    <name>dfs.datanode.http.address</name>
    <value>0.0.0.0:9864</value>
  </property>
</configuration>
```

- docker-compose.yml

```
services:
  namenode:
    image: apache/hadoop:3.4.3
    container_name: namenode
    hostname: namenode
    user: root
    command: >
      bash -c "
      mkdir -p /tmp/hadoop-data/namenode &&
      chown -R hadoop:hadoop /tmp/hadoop-data/namenode &&
      if [ ! -d /tmp/hadoop-data/namenode/current ]; then
        su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs namenode -format -force -nonInteractive';
      fi &&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs namenode'
      "
    ports:
      - "9870:9870"
      - "8020:8020"
```

```
    volumes:
      -          ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-
site.xml:ro
      -          ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-
site.xml:ro
      - namenode-data:/tmp/hadoop-data

  datanode:
    image: apache/hadoop:3.4.3
    container_name: datanode
    hostname: datanode
    user: root
    command: >
      bash -c "
      mkdir -p /tmp/hadoop-data/datanode &&
      chown -R hadoop:hadoop /tmp/hadoop-data/datanode
&&
      su -s /bin/bash hadoop -c '/opt/hadoop/bin/hdfs
datanode'
      "
    ports:
      - "9864:9864"
    volumes:
      -          ./conf/core-site.xml:/opt/hadoop/etc/hadoop/core-
site.xml:ro
      -          ./conf/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-
site.xml:ro
      - datanode-data:/tmp/hadoop-data
    depends_on:
      - namenode

volumes:
  namenode-data:
  datanode-data:
```

2. Validasi dan Penarikan Image

Perintah docker compose config dijalankan guna memastikan tidak ada kesalahan sintaks pada kode YAML. Setelah validasi sukses, *image* instalasi ditarik dari *registry* menggunakan perintah docker pull apache/hadoop:3.4.3.

3. Mengaktifkan Klaster Hadoop

   Sistem dinyalakan di latar belakang dengan perintah docker compose up -d. Verifikasi status *container* (NameNode dan DataNode) dilakukan menggunakan perintah docker compose ps -a.

4. Pembuatan Direktori HDFS

   Pembuatan direktori tugas di dalam HDFS dieksekusi dengan mendelegasikan perintah lewat *user* Hadoop, yaitu docker exec -it –user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/data dan /hasil.

5. Transfer Data dan Manipulasi File

   File data.txt ditransfer dari mesin *host* ke *container* via perintah docker cp, lalu diunggah permanen ke HDFS menggunakan utilitas -put. Pembacaan isi data diverifikasi menggunakan parameter -cat.

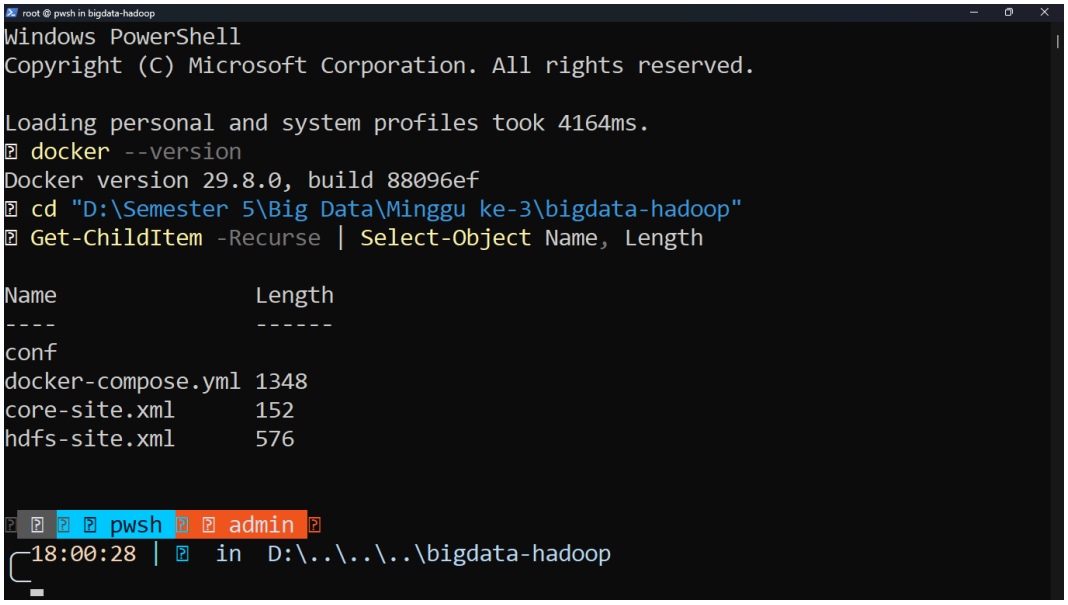
6. Manajemen *File* Terdistribusi

   Eksekusi penyalinan *file* dilakukan melalui parameter -cp, dilanjutkan dengan pengubahan nama *file* menggunakan parameter -mv, dan diakhiri dengan penghapusan *file backup* menggunakan parameter -rm.

7. Pengecekan Kesehatan Klaster

   Status kesehatan DataNode dan sisa kapasitas penyimpanan dicek secara menyeluruh menggunakan utilitas pelaporan hdfs dfsadmin -report.

3.2 Hasil Eksekusi dan Screenshot



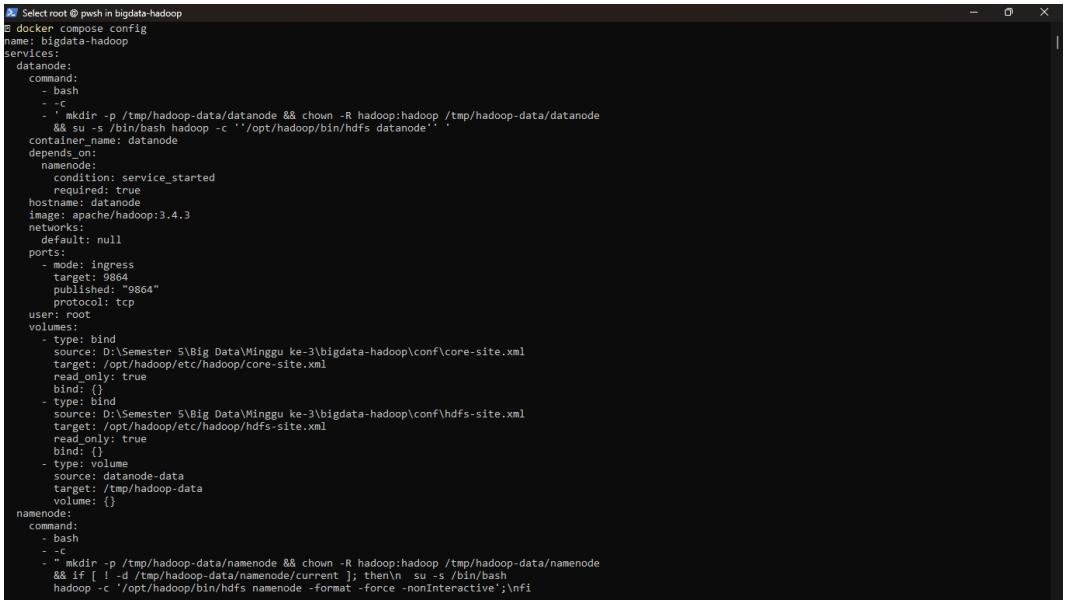
Gambar 3.2.1 - Menunjukkan struktur folder dan file yang berhasil dibuat.png

```
docker --version

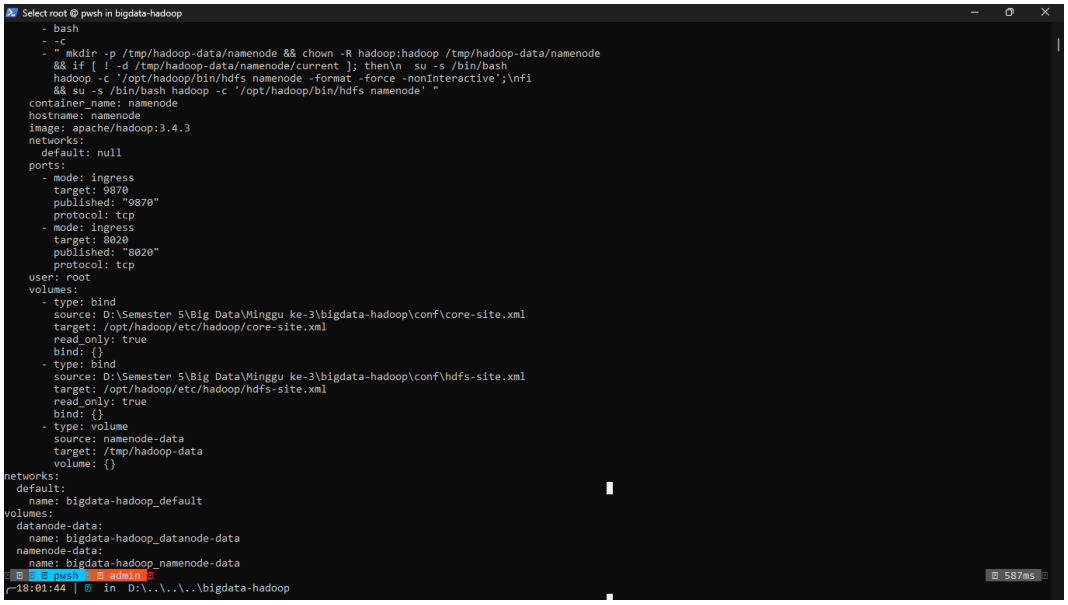
cd “D:\Semester 5\Big Data\Minggu ke-3\bigdata-hadoop”

Get-ChildItem -Recurse | Select-Object Name, Lenght
```

Berdasarkan hasil eksekusi pada terminal di atas, direktori kerja bigdata-hadoop beserta seluruh file pendukungnya telah berhasil dibentuk dengan sempurna. Pembuatan direktori ini dilakukan secara spesifik di dalam partisi tugas perkuliahan (D:\Semester 5\Big Data\Minggu ke-3). Keputusan pemilihan lokasi ini ditujukan semata-mata untuk mengoptimalkan manajemen proyek dan mencegah terjadinya penumpukan data pada partisi sistem operasi. Pembentukan struktur ini juga sepenuhnya didukung oleh Docker Compose yang bekerja mengandalkan sistem *relative path*, sehingga seluruh konfigurasi tetap berjalan secara optimal di mana pun direktori tersebut diletakkan. Pada direktori tersebut, file docker-compose.yml telah dipersiapkan bersama subdirektori conf yang menyimpan parameter krusial HDFS di dalam core-site.xml maupun hdfs-site.xml.



Gambar 3.2.2 - Validasi Docker Compose_1.png

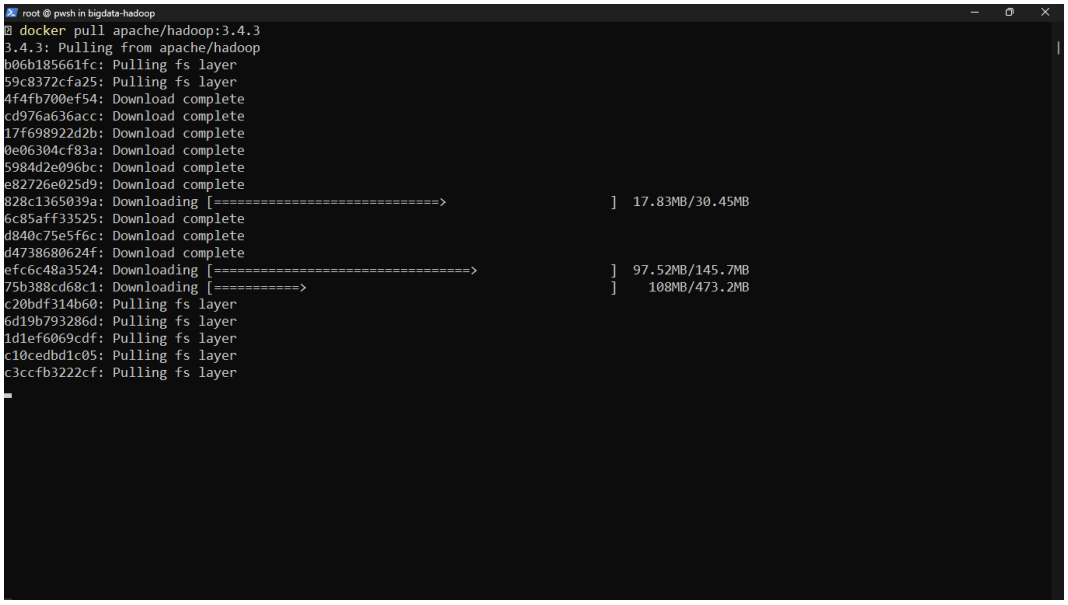


Gambar 3.2.3 - Validasi Docker Compose_2.png

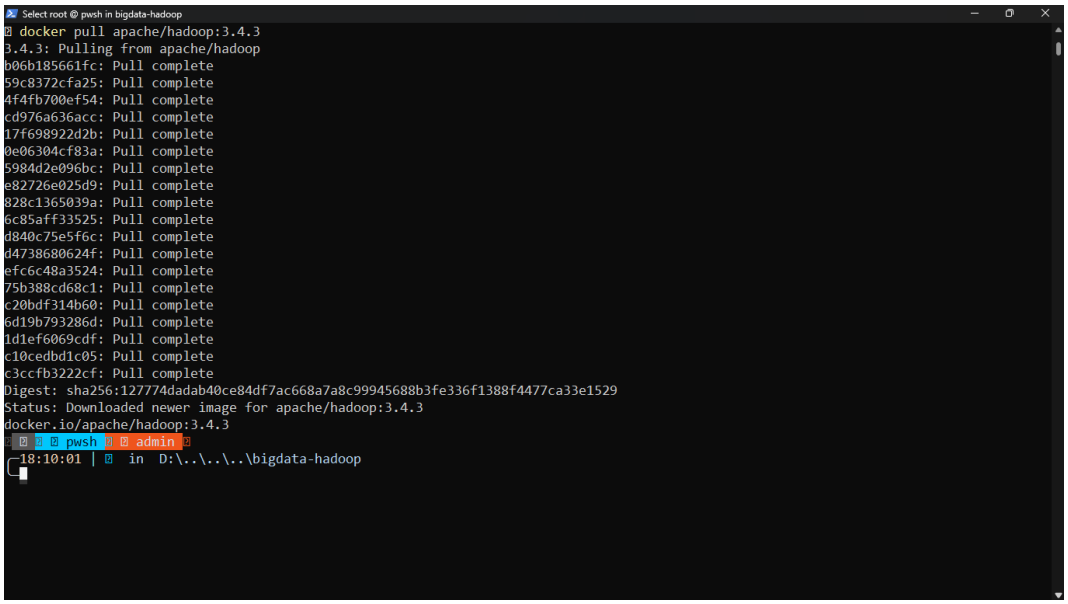
```
docker compose config
```

Proses validasi selanjutnya dijalankan menggunakan perintah docker compose config guna memverifikasi keabsahan struktur kode YAML yang telah dibuat. Tampilan layar terminal di atas memperlihatkan secara rinci hasil evaluasi *Docker Engine* yang tereksekusi dengan mulus tanpa adanya satupun

laporan *error* atau peringatan kegagalan. Keberhasilan validasi ini memberikan jaminan mutlak bahwa seluruh penulisan sintaks, definisi instruksi *bash* pada *container*, alokasi pemetaan *port*, serta pengaturan *volume data* telah tersusun secara presisi dan siap diimplementasikan untuk tahap *deployment* mesin virtual.



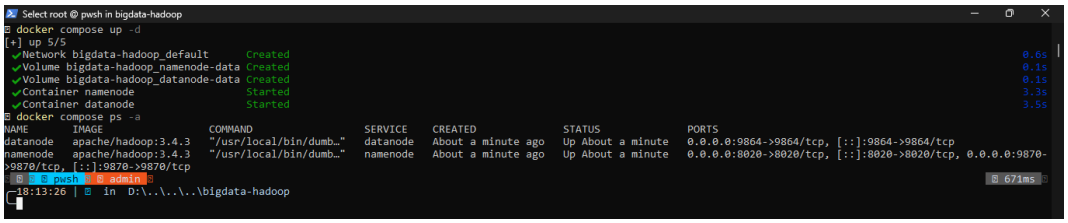
Gambar 3.2.4 - Proses Download Image_1.png



Gambar 3.2.5 - Proses Download Image_2.png

```
docker pull apache/hadoop:3.4.3
```

Langkah berikutnya berfokus pada penarikan (*pulling*) komponen instalasi Apache Hadoop dari repositori pusat Docker Hub menggunakan perintah docker pull apache/hadoop:3.4.3. Penetapan versi secara spesifik di angka 3.4.3 dilakukan dengan penuh kesengajaan demi memastikan keseragaman lingkungan pengembangan dengan instruksi modul praktikum. Pendekatan ini sekaligus berfungsi mengunci kestabilan sistem dari potensi kendala inkompatibilitas di kemudian hari. *Image* yang ditarik tersebut sudah membundel pustaka Java, sistem operasi dasar, serta fungsionalitas inti HDFS yang menjadi pondasi utama pembentukan klaster terdistribusi.

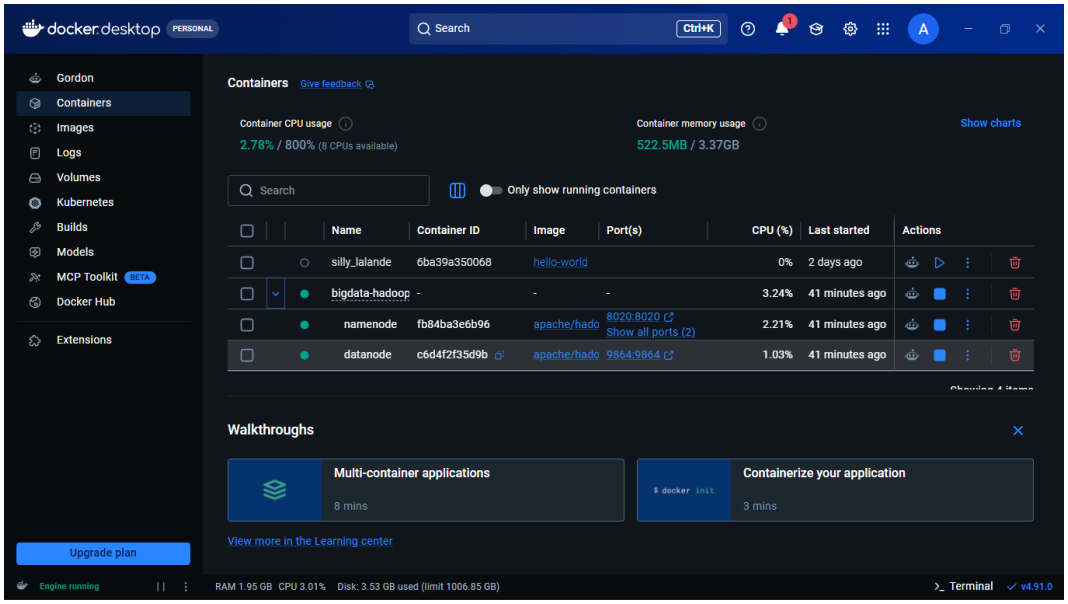


Gambar 3.2.6 - Menjalankan Cluster dan Verifikasi.png

```
docker compose up -d

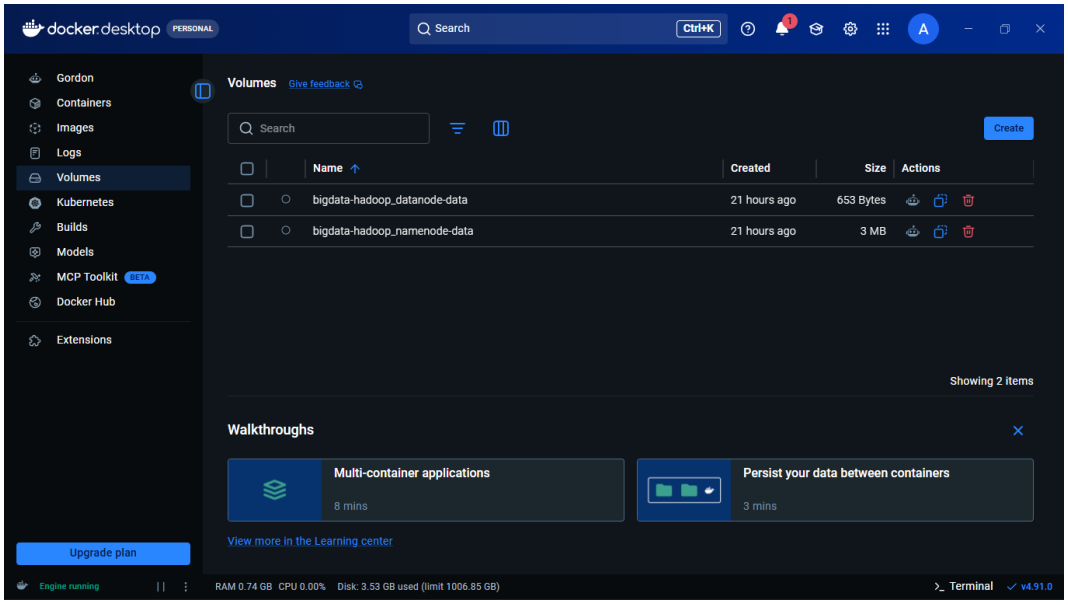
docker compose ps -a
```

Perintah docker compose up -d dijalankan untuk mengaktifkan seluruh layanan klaster secara serentak di latar belakang. Log yang tampil pada layar terminal mencatat lima objek berhasil dibentuk, mencakup satu *network*, dua *volume*, serta dua *container* yaitu namenode dan datanode yang keduanya berstatus Started. Untuk memastikan proses tersebut benar-benar stabil, perintah docker compose ps -a dieksekusi kembali sesaat setelahnya. Hasilnya memperlihatkan kedua *container* sudah berjalan dengan status Up selama kurang lebih satu menit, lengkap dengan pemetaan *port* 8020 dan 9870 pada namenode serta 9864 pada datanode. Kondisi ini menjadi bukti awal bahwa klaster Hadoop *single-node* sudah aktif dan siap menerima instruksi lanjutan.



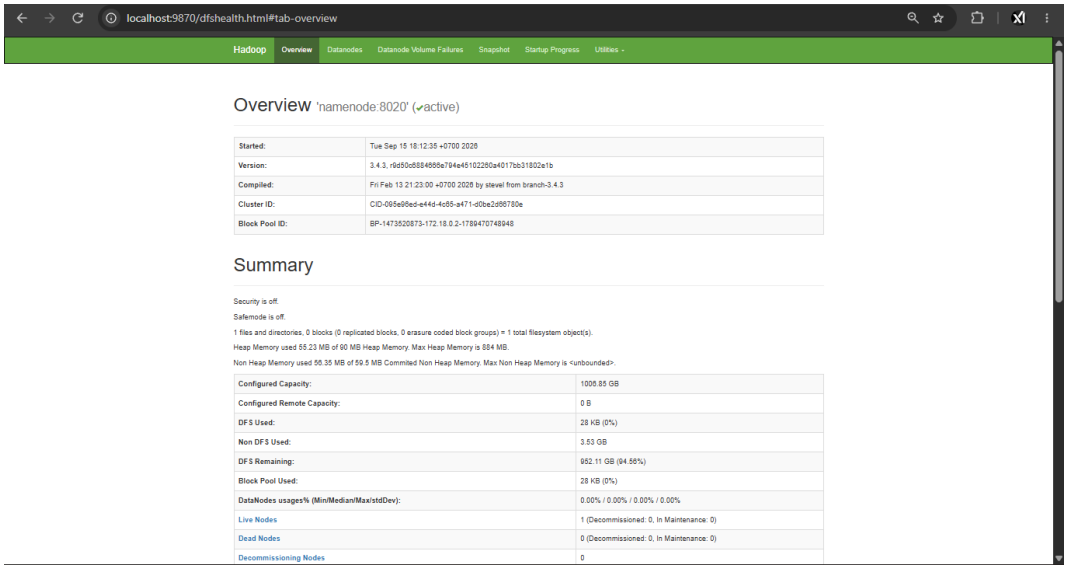
Gambar 3.2.7 - Informasi Docker Desktop Container.png

Tangkapan layar ini menampilkan panel Containers pada aplikasi Docker Desktop sebagai sarana pemantauan visual di luar terminal. Grup bigdata-hadoop terlihat menaungi dua *container* anggota, yakni namenode dan datanode, yang keduanya ditandai lingkaran hijau sebagai penanda status aktif. Informasi tambahan seperti pemakaian CPU sebesar 2,21% pada namenode dan 1,03% pada datanode turut ditampilkan, bersama daftar *image* apache/hadoop yang digunakan serta *port* publikasi masing-masing layanan. Pemanfaatan antarmuka grafis ini mempermudah proses pengawasan sumber daya klaster secara *real-time* tanpa harus terus-menerus mengetikkan perintah manual di *command line*.

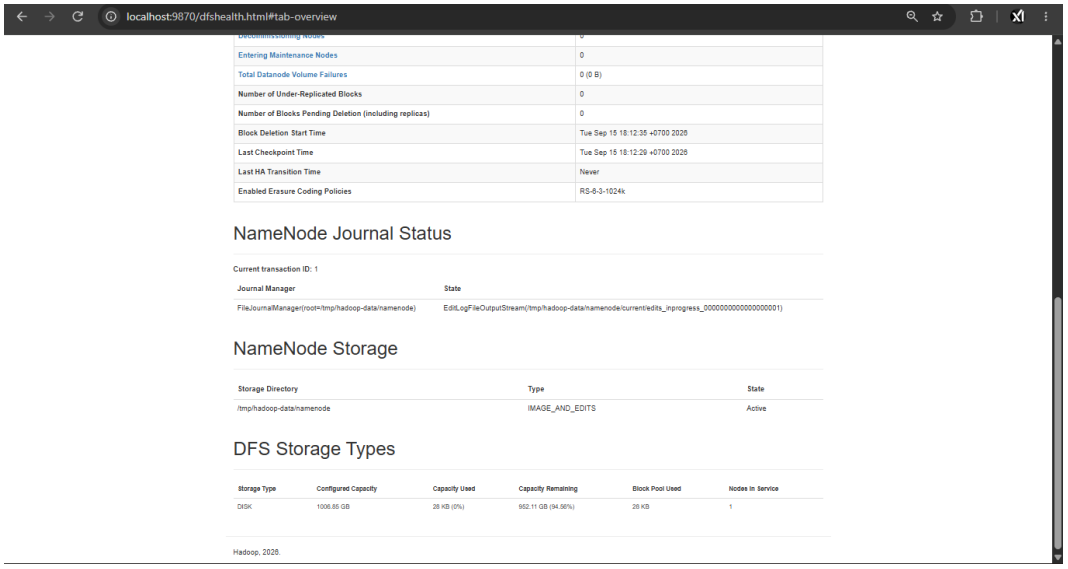


Gambar 3.2.8 - Menampilkan Informasi Volume pada Docker.png

Bagian Volumes pada Docker Desktop menampilkan dua *volume* yang tercipta otomatis mengikuti definisi pada docker-compose.yml, yaitu bigdata-hadoop_namenode-data dan bigdata-hadoop_datanode-data. Kedua *volume* tersebut berfungsi sebagai media penyimpanan permanen di luar siklus hidup *container*, sehingga data *metadata* pada namenode maupun data fisik pada datanode tetap tersimpan aman sekalipun *container*-nya suatu saat dihentikan atau dihapus. Ukuran *volume* namenode tercatat sebesar 3 MB, sedangkan *volume* datanode masih berukuran sangat kecil karena belum banyak blok data yang tersimpan di dalamnya pada tahap ini.



Gambar 3.2.9 - NameNode Web UI_1.png

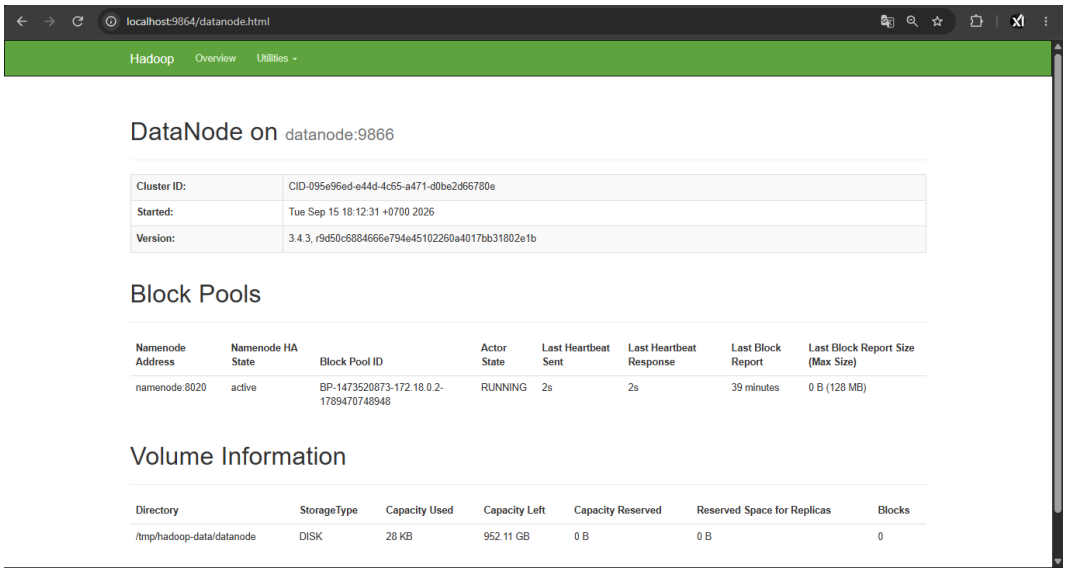


Gambar 3.2.10 - NameNode Web UI_2.png

Halaman *Web UI* NameNode diakses melalui peramban pada alamat localhost:9870/dfshealth.html. Bagian Overview memperlihatkan status namenode:8020 dalam kondisi active, lengkap dengan versi Hadoop 3.4.3 serta Cluster ID unik yang menjadi identitas klaster ini. Pada bagian Summary, tercatat bahwa fitur Safemode berada pada posisi off yang menandakan sistem sudah siap menerima operasi baca maupun tulis. Kapasitas yang dikonfigurasi tercatat sebesar 1006,85 GB dengan ruang tersisa mencapai 952,11 GB atau setara 94,56%,

sementara jumlah Live Nodes menunjukkan angka satu sesuai dengan skenario *single-node* pada praktikum ini.

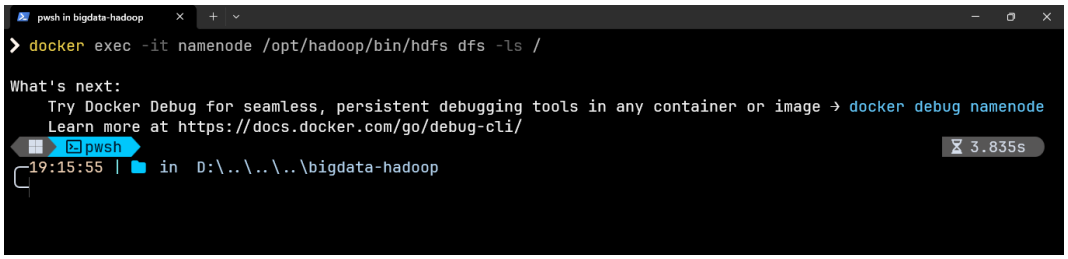
Informasi NameNode Journal Status memperlihatkan transaksi berjalan pada ID pertama, sedangkan bagian NameNode Storage menunjukkan direktori penyimpanan pada path /tmp/hadoop-data/namenode dengan tipe IMAGE_AND_EDITS berstatus Active. Tabel DFS Storage Types di bagian paling bawah merangkum kapasitas penyimpanan berjenis DISK secara lebih rinci, mencakup kapasitas terkonfigurasi, kapasitas terpakai, serta jumlah *node* yang sedang bertugas melayani klaster. Keseluruhan data pada halaman ini menegaskan bahwa metadata klaster tercatat dan terkelola dengan baik oleh NameNode.



Gambar 3.2.11 - DataNode Web UI.png

Halaman *Web UI* DataNode diakses melalui alamat localhost:9864/datanode.html dan menampilkan identitas datanode:9866 sebagai *node* pekerja tunggal pada klaster. Cluster ID yang tertera sama persis dengan yang tercatat pada NameNode, sehingga membuktikan bahwa datanode benar-benar tergabung pada klaster yang tepat. Tabel Block Pools menunjukkan hubungan aktif dengan namenode:8020 berstatus RUNNING dengan interval *heartbeat* terakhir hanya dua detik, menandakan komunikasi antar-*node* berjalan lancar. Pada bagian Volume Information, direktori /tmp/hadoop-data/datanode tercatat memiliki

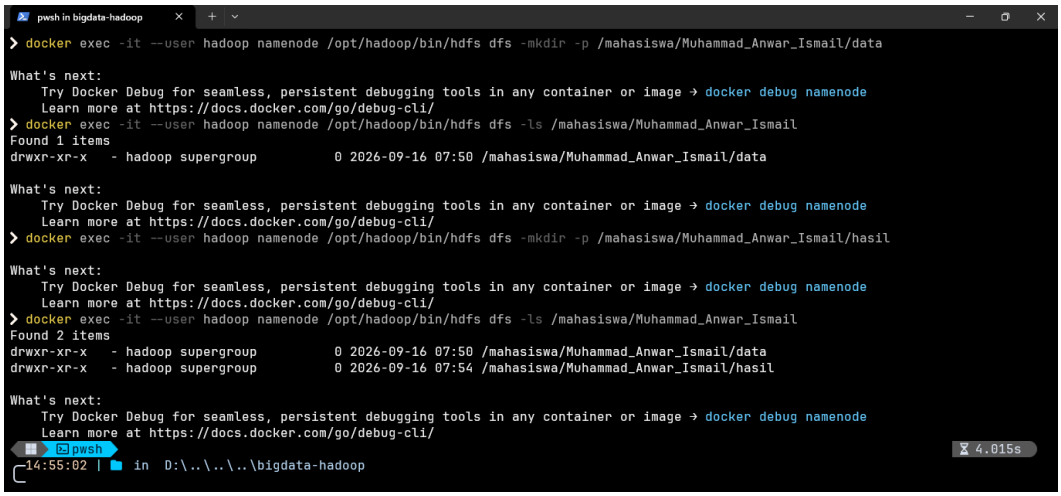
kapasitas tersisa sebesar 952,11 GB, memperkuat kondisi bahwa datanode siap menampung blok data dalam jumlah besar.



Gambar 3.2.12 - Melihat isi root HDFS.png

```
docker exec -it namenode /opt/Hadoop/bin/hdfs dfs -ls
```

Langkah praktik pengoperasian HDFS diawali dengan melakukan inspeksi terhadap direktori akar (*root*) menggunakan perintah hdfs dfs -ls /. Tangkapan layar di atas memperlihatkan eksekusi perintah yang langsung mengembalikan *prompt* kosong tanpa pesan *error*. Secara teknis, ketiadaan *output* ini memvalidasi bahwa mesin HDFS baru saja diformat secara bersih dan belum menampung struktur data maupun *file* apa pun, persis menyerupai perilaku utilitas direktori pada sistem operasi keluarga Unix yang tidak menampilkan data pada folder yang benar-benar kosong.



Gambar 3.2.13 - Membuat folder Praktikum, data, hasil dan verifikasi output direktori.png

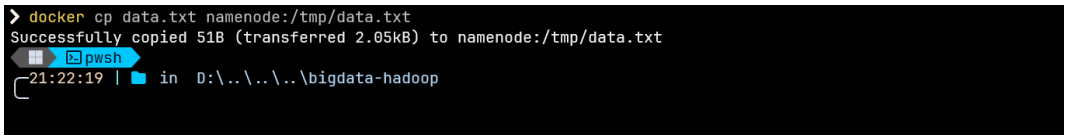
```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/data
```

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/hasil

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail
```

Rangkaian perintah hdfs dfs -mkdir -p dijalankan secara berurutan untuk membentuk struktur direktori kerja mahasiswa di dalam HDFS, dimulai dari folder data lalu dilanjutkan folder hasil di bawah path /mahasiswa/Muhammad_Anwar_Ismail. Setiap kali satu direktori selesai dibuat, perintah hdfs dfs -ls langsung disisipkan sebagai langkah verifikasi. Hasil akhirnya memperlihatkan dua direktori, yaitu data dan hasil, telah tercatat lengkap dengan hak akses drwxr-xr-x serta kepemilikan pada *user* hadoop dan *group* supergroup. Struktur folder ini nantinya menjadi lokasi utama penyimpanan seluruh *file* yang akan diproses pada tahapan praktikum selanjutnya.

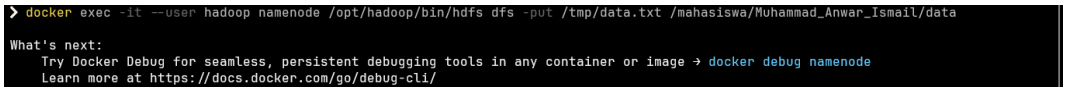


Gambar 3.2.14 - Salin file data.txt dari Windows ke dalam container NameNode.png

```
docker cp data.txt namenode:/tmp/data.txt
```

Perintah docker cp data.txt namenode:/tmp/data.txt digunakan untuk memindahkan *file* data.txt dari sistem lokal Windows menuju direktori sementara /tmp pada *container* namenode. Pesan konfirmasi Successfully copied yang muncul di layar menandakan transfer sebesar 51 byte berhasil dilakukan tanpa kendala. Tahapan ini menjadi jembatan penting sebelum *file* tersebut dapat diunggah ke dalam sistem HDFS, mengingat perintah -put pada HDFS hanya mampu membaca

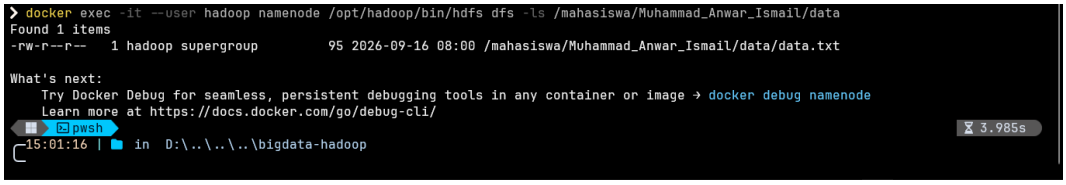
file yang sudah berada di dalam *filesystem container*, bukan langsung dari komputer *host*.



Gambar 3.2.15 - Pindahkan (upload) file tersebut dari container ke dalam HDFS.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -put /tmp/data.txt /mahasiswa/Muhammad_Anwar_Ismail/data
```

Setelah *file* data.txt tersedia di dalam *container* namenode, perintah hdfs dfs -put /tmp/data.txt /mahasiswa/Muhammad_Anwar_Ismail/data dijalankan untuk mengunggahnya secara permanen ke dalam sistem HDFS. Perintah ini tereksekusi tanpa memunculkan pesan *error* apa pun, yang pada dasarnya menjadi indikasi umum bahwa proses *upload* berjalan sukses sesuai kebiasaan perilaku *command line* berbasis Unix. Dengan selesainya tahap ini, *file* data.txt resmi tercatat sebagai bagian dari sistem penyimpanan terdistribusi dan siap diakses maupun diverifikasi pada langkah berikutnya.

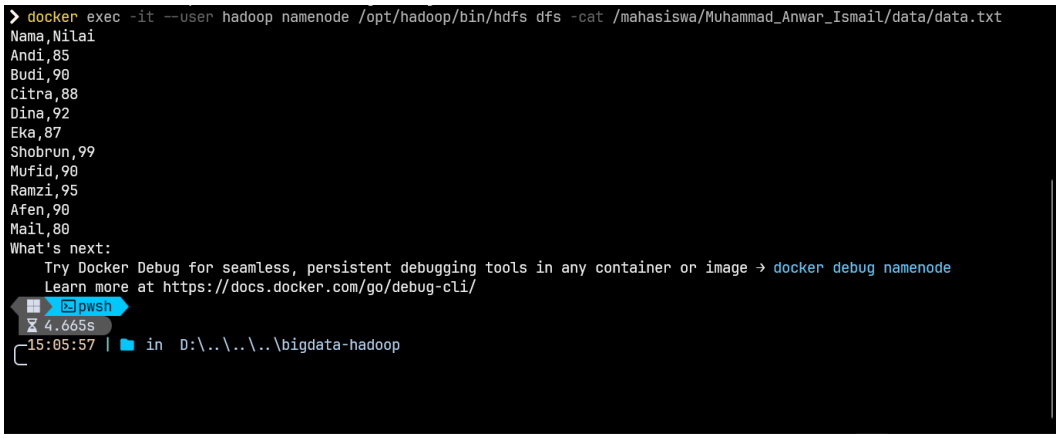


Gambar 3.2.16 - Tampilkan daftar file di direktori HDFS untuk memastikan file sudah ter-upload.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/data
```

Perintah hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/data dijalankan guna memvalidasi keberhasilan proses *upload* yang dilakukan pada tahapan sebelumnya. Terminal menampilkan keterangan Found 1 items, disertai rincian data.txt dengan hak akses -rw-r--r--, kepemilikan pada *user* hadoop dan *group* supergroup, serta ukuran *file* sebesar 95 byte lengkap dengan cap waktu pembuatannya. Kemunculan detail tersebut menjadi bukti konkret bahwa *file*

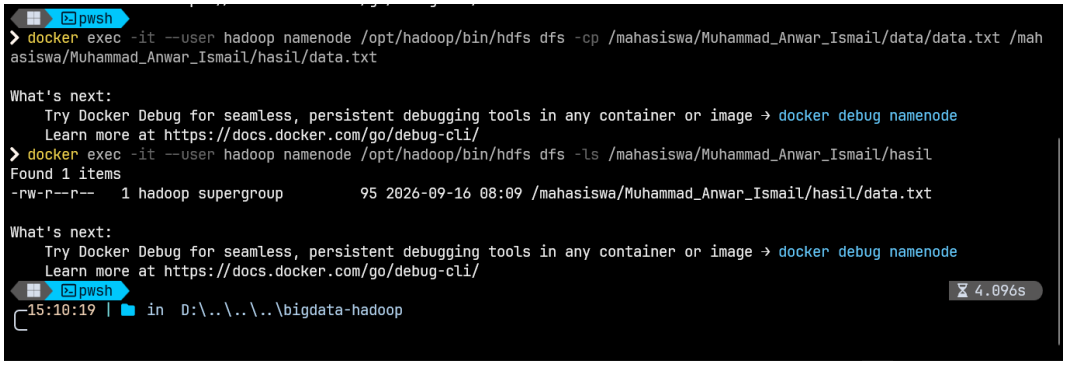
data.txt sudah benar-benar tersimpan di dalam sistem HDFS, bukan sekadar tercatat pada *log* proses *upload* semata. Tahap verifikasi semacam ini penting dilakukan sebagai kebiasaan baik sebelum melanjutkan ke pengujian isi *file* pada langkah berikutnya.



Gambar 3.2.17 - Tampilkan isi file data.txt yang ada di HDFS.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/data/data.txt
```

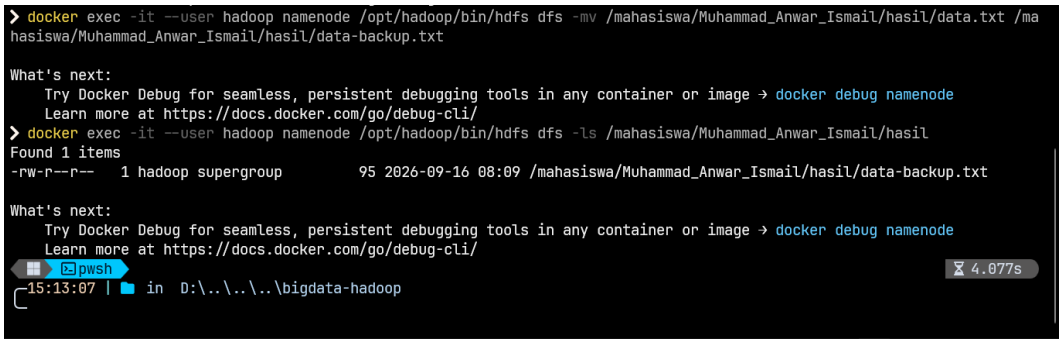
Setelah proses unggah selesai, perintah `hdfs dfs -ls` dijalankan kembali untuk mengecek isi direktori `/mahasiswa/Muhammad_Anwar_Ismail/data/data.txt`. *Output* di terminal memperlihatkan bahwa file data.txt sudah berhasil masuk ke HDFS, lengkap dengan informasi ukuran file dan waktu pembuatannya. Selanjutnya, untuk memastikan isi data tidak ada yang terpotong saat proses unggah, perintah `hdfs dfs -cat` digunakan. Perintah ini berfungsi untuk membaca dan menampilkan isi file teks secara langsung di layar terminal. Hasilnya, 10 baris data CSV berisi nama dan nilai mahasiswa berhasil ditampilkan dengan sempurna sesuai dengan file aslinya.



Gambar 3.2.18 - Buat salinan file (copy).png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cp
/mahasiswa/Muhammad_Anwar_Ismail/data/data.txt
/mahasiswa/Muhammad_Anwar_Ismail/hasil/data.txt
```

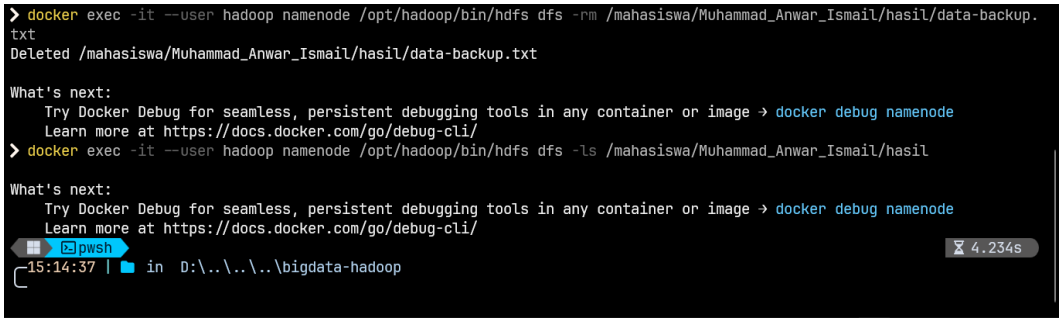
Perintah hdfs dfs -cp dijalankan untuk menggandakan *file* data.txt dari direktori data menuju direktori hasil, sehingga tercipta dua salinan identik pada lokasi berbeda di dalam HDFS. Proses penyalinan diverifikasi lewat perintah hdfs dfs -ls pada direktori hasil, dan hasilnya menunjukkan satu *file* bernama data.txt berukuran 95 byte dengan hak akses -rw-r--r-- telah tersimpan di sana. Kemampuan penggandaan *file* semacam ini sangat berguna dalam skenario pencadangan data maupun replikasi manual pada ekosistem HDFS.



Gambar 3.2.19 - Ubah nama (rename atau move) file salinan tersebut menjadi data-backup.txt.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mv
/mahasiswa/Muhammad_Anwar_Ismail/hasil/data.txt
/mahasiswa/Muhammad_Anwar_Ismail/hasil/data-backup.txt
```

Perintah hdfs dfs -mv dimanfaatkan untuk mengganti nama *file* data.txt yang berada di direktori hasil menjadi data-backup.txt. Perlu dipahami bahwa pada HDFS, operasi *rename* dan *move* sebenarnya memakai parameter yang sama karena keduanya sama-sama memindahkan referensi *path file* pada *metadata*. Verifikasi melalui perintah hdfs dfs -ls membuktikan bahwa *file* dengan nama baru data-backup.txt sudah tercatat pada direktori hasil, tetap dengan ukuran 95 byte yang identik dengan *file* asalnya, menandakan tidak ada data yang rusak maupun hilang selama proses pengubahan nama berlangsung.



Gambar 3.2.20 - Hapus (delete) file salinan yang baru saja di-rename tersebut.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -rm /mahasiswa/Muhammad_Anwar_Ismail/hasil/data-backup.txt
```

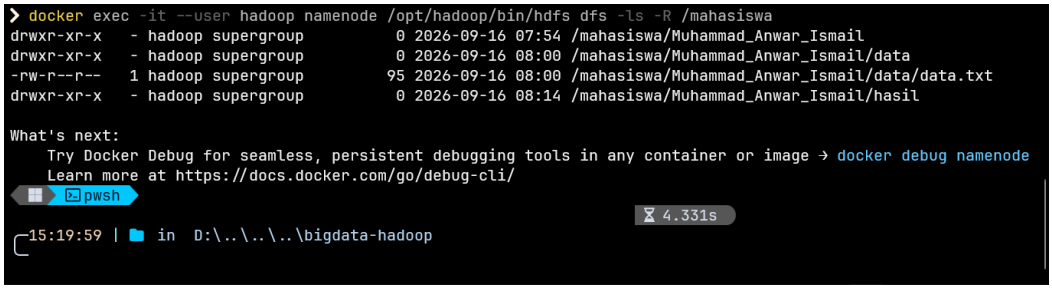
Perintah hdfs dfs -rm dieksekusi untuk menghapus *file* data-backup.txt dari direktori hasil. Sistem HDFS langsung merespons dengan pesan Deleted yang mengonfirmasi bahwa *file* tersebut telah berhasil dihapus secara permanen dari struktur penyimpanan. Langkah verifikasi lanjutan menggunakan hdfs dfs -ls pada direktori hasil tidak lagi menampilkan satu pun *item*, membuktikan direktori tersebut sudah kembali kosong sepenuhnya. Rangkaian operasi ini melengkapi siklus manajemen *file* dasar pada HDFS mulai dari penyalinan, pengubahan nama, hingga penghapusan.



Gambar 3.2.21 - Mengecek status DataNode dan kapasitas HDFS.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

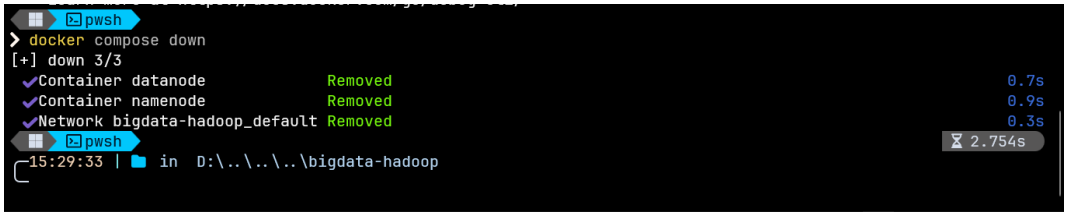
Perintah hdfs dfsadmin -report dijalankan untuk menghasilkan laporan menyeluruh mengenai kesehatan klaster. Laporan tersebut mencatat kapasitas total sebesar 1006,85 GB dengan pemakaian DFS hanya 44 KB atau setara 0,00%, serta tidak ditemukan satu pun blok rusak maupun hilang pada kategori Replicated Blocks. Bagian Live datanodes menegaskan satu *node* bernama datanode berada pada status Normal dengan sisa kapasitas mencapai 94,56%. Laporan ini menjadi alat diagnostik utama bagi administrator untuk memastikan seluruh komponen klaster berfungsi normal sebelum maupun sesudah rangkaian operasi *file* dilakukan.



Gambar 3.2.22 - Verifikasi akhir (Menampilkan seluruh isi folder mahasiswa).png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls -R /mahasiswa
```

Perintah hdfs dfs -ls -R /mahasiswa dijalankan untuk menampilkan struktur direktori secara rekursif sebagai bentuk pengecekan akhir. Hasilnya memperlihatkan seluruh hierarki folder milik Muhammad_Anwar_Ismail, mulai dari direktori data yang masih menyimpan *file* data.txt berukuran 95 byte, hingga direktori hasil yang tampak kosong karena *file* di dalamnya sudah dihapus pada tahapan sebelumnya. Tampilan rekursif ini secara ringkas merangkum seluruh jejak operasi manajemen *file* yang telah dipraktikkan, mulai dari pembuatan direktori, *upload*, penyalinan, pengubahan nama, sampai penghapusan.



Gambar 3.2.23 - Mematikan Docker Container.png

```
docker compose down
```

Sebagai penutup rangkaian praktikum, perintah docker compose down dijalankan untuk menghentikan sekaligus membersihkan seluruh sumber daya yang telah dibangun. Log yang tampil mencatat *container* datanode dan namenode berhasil dihapus (Removed), disusul penghapusan *network* bigdata-hadoop_default, dengan keseluruhan proses tuntas dalam waktu kurang dari tiga detik. Langkah ini penting dilakukan agar sumber daya komputer tidak terus

terpakai secara sia-sia setelah seluruh rangkaian pengujian selesai dikerjakan, sekaligus menjaga kebersihan lingkungan *Docker* pada perangkat yang digunakan.

- CHECKLIST KEBERHASILAN PRAKTIKUM

[√] WSL 2 aktif

[√] Docker Desktop berjalan

[√] docker run hello-world berhasil

[√] Image apache/hadoop:3.4.3 berhasil di-download

[√] NameNode berstatus Up

[√] DataNode berstatus Up

[√] localhost:9870 dapat dibuka

[√] Direktori /mahasiswa/Muhammad_Anwar_Ismail/data dan /hasil berhasil dibuat

[√] File berhasil di-upload ke HDFS

[√] File dapat dibaca dengan -cat

[√] File berhasil di-copy/move/delete

[√] dfsadmin -report menampilkan DataNode aktif

3.3 Jawaban Pertanyaan Analisis

10. Apa fungsi NameNode dan DataNode?

    NameNode berfungsi secara eksklusif sebagai otak pengelola tata letak file dan struktur direktori HDFS. Sementara itu, DataNode bertugas murni sebagai tempat node yang menyediakan ruang penyimpanan fisik untuk potongan-potongan data tersebut..

11. Mengapa NameNode menyimpan metadata, sedangkan DataNode menyimpan block data?

    Pembagian peran ini dilakukan semata-mata demi efisiensi performa. NameNode bertindak layaknya daftar isi buku yang butuh kecepatan akses super tinggi lewat RAM. Jika NameNode ikut memikul beban menampung file aslinya, memori pasti akan langsung jebol dan seluruh sistem bakal lumpuh.

Makanya, tugas berat I/O data mentahnya diserahkan penuh kepada barisan DataNode.

12. Apa yang dimaksud dengan block pada HDFS?

    Block merupakan kepingan atau satuan terkecil penyimpanan di dalam ekosistem HDFS. Setiap kali ada file raksasa yang masuk, sistem otomatis mencacahnya menjadi beberapa bagian kecil (biasanya berukuran 128 MB per keping), lalu menyebarkannya ke berbagai titik DataNode.

13. Apa fungsi replication factor?

    Fitur ini bertugas mengatur jumlah salinan cadangan dari setiap kepingan data. Tujuannya sangat jelas yakni untuk menjamin keamanan data. Seandainya ada satu server yang mendadak *down* atau rusak, kita tetap punya jaminan akses ke data utuh lewat salinannya di mesin lain.

14. Mengapa *replication factor* praktikum ditetapkan 1?

    Kita menyetel nilainya di angka 1 karena simulasi cluster Docker kita dirancang sangat minimalis dengan satu DataNode aktif saja. Keputusan ini sangat membantu menghemat pemakaian RAM laptop kita. Apabila kita paksa di angka yang lebih tinggi, sistem Hadoop pasti akan terus melontarkan *error* karena kebingungan mencari mesin cadangan.

15. Apa yang terjadi jika DataNode mati ketika *replication factor* = 1?

    Akses menuju data tersebut akan terputus total. Mengingat kita sama sekali tidak punya salinan cadangan, kerusakan permanen pada node tunggal itu otomatis membuat seluruh data kita lenyap tak bersisa.

16. Apa perbedaan path C:... dengan /... pada HDFS?

    Path C:\... murni mewakili lokasi fisik pada sistem operasi lokal Windows kita. Di sisi lain, path /... merupakan struktur direktori virtual yang terisolasi dan sepenuhnya dikendalikan oleh NameNode HDFS. Kita wajib menggunakan perintah terminal khusus untuk bisa menembus dan melihat isi area HDFS tersebut.

17. Mengapa HDFS sesuai untuk penyimpanan Big Data?

    Sistem ini terbukti sangat tangguh menangani ukuran file luar biasa besar berkat kemampuannya memecah beban kerja. HDFS dirancang berjalan mulus di

spesifikasi komputer standar, sangat gampang diperluas kapasitasnya, dan punya jaminan keamanan data yang kuat berkat fitur replikasi otomatisnya.

18. Bagaimana perubahan jumlah DataNode memengaruhi HDFS?

    Semakin banyak DataNode yang kita gabungkan, ruang tampung total otomatis melesat naik. Kinerja proses baca dan tulis juga jauh lebih cepat karena sistem mengerjakannya secara bersamaan, sekaligus membuka peluang bagi kita untuk memperbanyak jumlah salinan pengaman data.

BAB IV
KESIMPULAN

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi Apache Hadoop dan HDFS dapat berjalan optimal di lingkungan sistem operasi Windows menggunakan bantuan WSL 2 dan Docker Desktop. Klaster berskala Tunggal (ingle-node) yang terdiri atas satu NameNode dan satu DataNode berhasil dibangun dan dikonfigurasi dengan baik menggunakan Docker Compose.

Seluruh antarmuka *Web-UI* sukses diakses untuk memantau status Kesehatan *node* secara *real-time*. Selain itu, eksekusi perintah dasar HDFS mulai dari membuat direktori (mkdir), mengunggah data (mengupload), membaca isi teks (cat), menyalin (cp), mengubah nama (mv), hingga menghapus file (rm) yang telah berhasil dilakukan sepenuhnya tanpa error. Pemahaman mendasar mengenai pengoperasian baris perintah (*command line)* dan manajemen file terdistribusi ini menjadi pondasi yang sangat penting sebelum melangkah ke materi pemrosesan Big Data lanjutan seperti YARN, MapReduce, maupun Apache Spark.

LAMPIRAN A
COMMAND UTAMA

JENIS	COMMAND UTAMA
Validasi Docker	docker compose config
Download Image	docker pull apache/hadoop:3.4.3
Menyalakan Klaster	docker compose up -d
Melihat Status Container	docker compose ps -a
Mematikan Klaster	docker compose down
List Direktori HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /
Membuat Direktori HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/data
Upload File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -put /tmp/data.txt /mahasiswa/Muhammad_Anwar_Ismail/data/
Membaca File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/data/data.txt
Copy File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cp /mahasiswa/Muhammad_Anwar_Ismail/data/data.txt /mahasiswa/Muhammad_Anwar_Ismail/hasil/data.txt

Rename File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mv /mahasiswa/Muhammad_Anwar_Ismail/hasil/data.txt /mahasiswa/Muhammad_Anwar_Ismail/hasil/data-backup.txt
Hapus File HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -rm /mahasiswa/Muhammad_Anwar_Ismail/hasil/data-backup.txt
Laporan Status HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report

LAMPIRAN B

PORT YANG DIGUNAKAN

Port	Komponen	Digunakan
9870	NameNode	Untuk mengakses halaman *Web UI* (antarmuka grafis) milik NameNode via *browser*.
8020	NameNode	Sebagai *port* komunikasi layanan internal HDFS (RPC) milik NameNode.
9864	DataNode	Untuk mengakses halaman *Web UI* milik DataNode.